AICE3001/AICE6002: Mathematics of Machine Learning

Subsidiary Notes — Lecture 27: Optimal Transport and the Wasserstein Distance

Adam Prugel-Bennett

November 2, 2026

1 Keywords

- Wasserstein distance, optimal transport, duality, Lipschitz functions, WGANs, Sinkhorn, FID

2 Main Points

2.1 Comparing Distributions

- Many machine learning tasks need a measure of how different two probability distributions are, e.g. to make the samples of a generative model look like the data
- The most common is the KL-divergence

$$\mathrm{KL}(p\|q) = \int p(\boldsymbol{x})\, \log\left(\frac{p(\boldsymbol{x})}{q(\boldsymbol{x})}\right)\, \mathrm{d}\boldsymbol{x}$$

  - it is non-negative and zero only when $p = q$
  - it is not a distance (it is not symmetric and does not satisfy the triangular inequality), but that is not the main problem
  - if $q(\boldsymbol{x}) = 0$ where $p(\boldsymbol{x}) \neq 0$ it is infinite, so two distributions that look very similar can have a huge (or infinite) KL-divergence
- Real data such as images lie close to a thin, low-dimensional manifold in a very high-dimensional space. Two such distributions can be close without overlapping at all, and then the KL-divergence tells us nothing useful

2.2 Optimal Transport

- The *Wasserstein* or *Earth Mover's* distance is the least total work needed to move the probability mass of p so that it becomes q, where work is mass times the distance it is moved
- A *transportation policy* $\gamma(\boldsymbol{x}, \boldsymbol{y}) \geq 0$ says how much mass moves from $\boldsymbol{x}$ to $\boldsymbol{y}$. Everything that leaves $\boldsymbol{x}$ must add up to $p(\boldsymbol{x})$, and everything that arrives at $\boldsymbol{y}$ to $q(\boldsymbol{y})$:

$$\int \gamma(\boldsymbol{x}, \boldsymbol{y})\, \mathrm{d}\boldsymbol{y} = p(\boldsymbol{x}), \qquad \int \gamma(\boldsymbol{x}, \boldsymbol{y})\, \mathrm{d}\boldsymbol{x} = q(\boldsymbol{y})$$

  so γ is a joint distribution with marginals p and q; we call the set of these $\Lambda(p, q)$
- With a distance $d(\boldsymbol{x}, \boldsymbol{y}) = \|\boldsymbol{x} - \boldsymbol{y}\|$ the cost of a policy is $C(\gamma) = \mathbb{E}_\gamma[d(\boldsymbol{x}, \boldsymbol{y})]$ and

$$W(p, q) = \min_{\gamma \in \Lambda(p,q)} \mathbb{E}_\gamma[d(\boldsymbol{x}, \boldsymbol{y})]$$

- More generally, with cost d^k, $W_k(p,q) = \big(\min_\gamma \mathbb{E}_\gamma\big[d(\boldsymbol{x},\boldsymbol{y})^k\big]\big)^{1/k}$. The lecture's W is W_1; FID (below) uses W_2
- **One dimension.** Here there is a closed form in terms of the cumulative distribution functions,
$$W_1(p,q) = \int_{-\infty}^{\infty} \big|F_p(x) - F_q(x)\big|\,\mathrm{d}x$$
the area between the two curves. For two samples of the same size, sort both and match them in order: $W_1 = \frac{1}{n}\sum_i |x_{(i)} - y_{(i)}|$. In more than one dimension there is no such shortcut

2.3 Duality

- For discrete distributions the unknowns are the numbers $\gamma(\boldsymbol{x}_i,\boldsymbol{y}_j) \geq 0$, the objective $\sum_{i,j} d(\boldsymbol{x}_i,\boldsymbol{y}_j)\,\gamma(\boldsymbol{x}_i,\boldsymbol{y}$ is linear and so are the constraints: it is a linear programme, like those in the constrained-optimisation lecture
- With Lagrange multipliers $\alpha(\boldsymbol{x}_i)$ and $\beta(\boldsymbol{y}_j)$ for the two sets of constraints the Lagrangian rearranges to
$$\mathcal{L} = \sum_i \alpha(\boldsymbol{x}_i)\,p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)\,q(\boldsymbol{y}_j) - \sum_{i,j} \gamma(\boldsymbol{x}_i,\boldsymbol{y}_j)\big(\alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) - d(\boldsymbol{x}_i,\boldsymbol{y}_j)\big)$$
Because $\gamma \geq 0$, the dual problem is to maximise $\sum_i \alpha(\boldsymbol{x}_i)\,p(\boldsymbol{x}_i) + \sum_j \beta(\boldsymbol{y}_j)\,q(\boldsymbol{y}_j)$ subject to $\alpha(\boldsymbol{x}_i) + \beta(\boldsymbol{y}_j) \leq d(\boldsymbol{x}_i,\boldsymbol{y}_j)$ for all i, j. For continuous distributions the sums become integrals and α, β become functions
- **Eliminating** β. We need $\alpha(\boldsymbol{x}) + \beta(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y})$ for all $\boldsymbol{x}$, $\boldsymbol{y}$, and want β as large as possible. Taking $\boldsymbol{x} = \boldsymbol{y}$, where $d(\boldsymbol{y},\boldsymbol{y}) = 0$, gives $\beta(\boldsymbol{y}) \leq -\alpha(\boldsymbol{y})$: so $-\alpha$ is the largest β could possibly be
- Try that largest value, $\beta = -\alpha$. The constraint becomes
$$\alpha(\boldsymbol{x}) - \alpha(\boldsymbol{y}) \leq d(\boldsymbol{x},\boldsymbol{y}) \quad \text{for all } \boldsymbol{x},\boldsymbol{y}$$
which says α is *1-Lipschitz*. For any such α, $\beta = -\alpha$ satisfies every constraint, and no other β can do better because $\beta \leq -\alpha$ always
- We lose nothing by restricting to 1-Lipschitz α (*Kantorovich–Rubinstein*). Take any feasible pair (α,β) and let $f(\boldsymbol{x}) = \min_{\boldsymbol{y}}\big(d(\boldsymbol{x},\boldsymbol{y}) - \beta(\boldsymbol{y})\big)$. The constraint gives $f \geq \alpha$, so (f,β) is still feasible and the objective is no smaller. f is a minimum of functions that are each 1-Lipschitz in $\boldsymbol{x}$ (by the triangular inequality), so f is 1-Lipschitz, and by the previous point we can then replace β by $-f$ without making the objective smaller
- This gives the *Kantorovich–Rubinstein* duality
$$W(p,q) = \max_{\alpha\ \text{1-Lipschitz}} \Big(\mathbb{E}_p[\alpha(\boldsymbol{X})] - \mathbb{E}_q[\alpha(\boldsymbol{X})]\Big)$$
Instead of the huge object $\gamma(\boldsymbol{x},\boldsymbol{y})$ we need one function $\alpha(\boldsymbol{x})$, and the expectations can be estimated from samples
- For differentiable α, being 1-Lipschitz is the same as $\|\boldsymbol{\nabla}\alpha(\boldsymbol{x})\| \leq 1$ everywhere: the function can change no faster than its input

2.4 Applications

- **Wasserstein GANs.** A GAN has a generator $G(\boldsymbol{z}, \boldsymbol{w}_G)$ that turns noise $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ into images, and a discriminator trained to tell them from real images. GANs are hard to train: a discriminator that is too good gives the generator no useful gradient. A Wasserstein GAN trains the generator to minimise W between the data distribution and the generator's distribution,

  $$\min_{\boldsymbol{w}_G} \max_{\boldsymbol{w}_\alpha} \Big(\frac{1}{|\mathcal{B}|} \sum_{\boldsymbol{x} \in \mathcal{B}} \alpha(\boldsymbol{x}, \boldsymbol{w}_\alpha) - \frac{1}{n} \sum_{i=1}^{n} \alpha(G(\boldsymbol{z}_i, \boldsymbol{w}_G), \boldsymbol{w}_\alpha) \Big)$$

  with expectations estimated from a minibatch $\mathcal{B}$ of real images and n generated ones

  - the *critic* α replaces the discriminator; it still gives a gradient when the distributions do not overlap
  - it should be 1-Lipschitz, which in practice is only roughly enforced: the original WGAN clipped the weights, WGAN-GP penalises $\|\nabla \alpha\|$ away from 1, and spectral normalisation scales each weight matrix to spectral norm 1 (with 1-Lipschitz activation functions the network is then at most 1-Lipschitz)

- **Sinkhorn.** Solving the linear programme for n points costs roughly $O(n^3)$. Cuturi (2013) added an entropy term,

  $$\min_{\gamma} \sum_{i,j} d_{ij}\, \gamma_{ij} + \epsilon \sum_{i,j} \gamma_{ij} \log \gamma_{ij}$$

  with the same constraints. The solution has the form $\gamma_{ij} = u_i\, K_{ij}\, v_j$ with $K_{ij} = \mathrm{e}^{-d_{ij}/\epsilon}$ (Exercise), and $\boldsymbol{u}$, $\boldsymbol{v}$ are found by alternately rescaling rows and columns until the marginals are right,

  $$\boldsymbol{v} \leftarrow \boldsymbol{q} \oslash (\mathbf{K}^{\mathsf{T}} \boldsymbol{u}), \qquad \boldsymbol{u} \leftarrow \boldsymbol{p} \oslash (\mathbf{K} \boldsymbol{v})$$

  (Sinkhorn's matrix scaling algorithm; $\oslash$ is element-by-element division)

  - each step is two matrix–vector products, so it is fast on GPUs and differentiable
  - as $\epsilon \to 0$ the cost approaches the exact transport cost; larger ϵ converges faster but blurs the plan (see the experiment)

- **FID.** For Gaussians, W_2 has a closed form

  $$W_2^2 = \|\boldsymbol{\mu}_1 - \boldsymbol{\mu}_2\|^2 + \operatorname{Tr}\Big(\boldsymbol{\Sigma}_1 + \boldsymbol{\Sigma}_2 - 2\big(\boldsymbol{\Sigma}_1 \boldsymbol{\Sigma}_2\big)^{1/2} \Big)$$

  The *Fr\'echet Inception Distance* (Heusel et al., 2017) fits Gaussians to a pretrained network's features of real and of generated images and reports this quantity. It is the standard score for image generators, GANs and diffusion models alike (lower is better), though it only compares the means and covariances of the features

3 Exercises

3.1 Disjoint supports

- p puts probability $\frac{1}{2}$ on each of $x = 0$ and $x = 2$; q puts probability 1 on $x = 1$. Find $W_1(p, q)$, $\mathrm{KL}(p \| q)$ and $\mathrm{KL}(q \| p)$.

3.2 Shifting a distribution

- Let p be uniform on $[0, 1]$ and q uniform on $[a, 1 + a]$ with $0 < a < 1$. Use the one-dimensional formula to find $W_1(p, q)$. What is $\mathrm{KL}(p \| q)$?

3.3 FID in one dimension

- Show that for one-dimensional Gaussians the W_2^2 formula reduces to $(\mu_1 - \mu_2)^2 + (\sigma_1 - \sigma_2)^2$.

3.4 The Sinkhorn solution

- Write the Lagrangian of the entropic problem with multipliers α_i and β_j for the two sets of constraints. Set its derivative with respect to γ_{ij} to zero and show $\gamma_{ij} = u_i \, \mathrm{e}^{-d_{ij}/\epsilon} \, v_j$. What are u_i and v_j?

4 Experiments

4.1 Exact transport, sorting and Sinkhorn

- For two samples on a line, compare the linear programme, the sort-and-match formula and Sinkhorn with several ϵ

```
import numpy as np
from scipy.optimize import linprog

rng = np.random.default_rng(0)
n = 8
x = np.sort(rng.normal(0, 1, n)); y = np.sort(rng.normal(1, 0.5, n))
p = np.full(n, 1/n); q = np.full(n, 1/n)
D = np.abs(x[:, None] - y[None, :])                  # cost |x - y|

# exact optimal transport as a linear programme
A = np.vstack([np.kron(np.eye(n), np.ones(n)), np.kron(np.ones(n), np.eye(n))])
res = linprog(D.ravel(), A_eq=A, b_eq=np.r_[p, q], bounds=(0, None))
print(f"LP:               W1 = {res.fun:.4f}")

# one dimension, equal weights: sort both samples and match in order
print(f"sort and match:   W1 = {np.mean(np.abs(x - y)):.4f}")

# Sinkhorn: entropic OT, gamma = diag(u) K diag(v)
for eps in [0.5, 0.1, 0.02]:
    K = np.exp(-D / eps); u = np.ones(n)
    for _ in range(5000):
        v = q / (K.T @ u)
        u = p / (K @ v)
    G = u[:, None] * K * v[None, :]
    print(f"Sinkhorn eps={eps:4}: cost = {np.sum(G * D):.4f}  (marginal error {np.abs(G.sum
```

- The linear programme and sorting both give $W_1 = 0.2909$. Sinkhorn gives 0.436, 0.312 and 0.293 for $\epsilon = 0.5$, 0.1 and 0.02: it approaches the exact cost as ϵ shrinks, with the marginals matched exactly

5 Answers

5.1 Disjoint supports

- Each half unit of mass moves a distance 1, so $W_1 = 1$
- $\mathrm{KL}(p\|q) = \infty$, because $q(0) = q(2) = 0$ where $p > 0$; $\mathrm{KL}(q\|p) = \infty$ too, because $p(1) = 0$. The distributions are close but have no overlap, which is exactly where the Wasserstein distance is useful

5.2 Shifting a distribution

- $F_p(x) - F_q(x) = x$ on $[0, a]$, a on $[a, 1]$ and $1 + a - x$ on $[1, 1 + a]$, so $W_1 = \frac{a^2}{2} + a(1 - a) + \frac{a^2}{2} = a$: the distance the mass was shifted
- $\mathrm{KL}(p\|q) = \infty$, since $q = 0$ on $[0, a)$ where $p > 0$

5.3 FID in one dimension

- $\mathrm{Tr}(\sigma_1^2 + \sigma_2^2 - 2(\sigma_1^2\sigma_2^2)^{1/2}) = \sigma_1^2 + \sigma_2^2 - 2\sigma_1\sigma_2 = (\sigma_1 - \sigma_2)^2$

5.4 The Sinkhorn solution

- The Lagrangian is

$$\mathcal{L} = \sum_{ij} d_{ij}\gamma_{ij} + \epsilon \sum_{ij} \gamma_{ij} \log \gamma_{ij} - \sum_i \alpha_i \Big(\sum_j \gamma_{ij} - p_i \Big) - \sum_j \beta_j \Big(\sum_i \gamma_{ij} - q_j \Big)$$

- $\partial\mathcal{L}/\partial\gamma_{ij} = d_{ij} + \epsilon(\log \gamma_{ij} + 1) - \alpha_i - \beta_j = 0$, so $\gamma_{ij} = \mathrm{e}^{(\alpha_i - \epsilon/2)/\epsilon}\, \mathrm{e}^{-d_{ij}/\epsilon}\, \mathrm{e}^{(\beta_j - \epsilon/2)/\epsilon}$: $u_i = \mathrm{e}^{\alpha_i/\epsilon - 1/2}$ and $v_j = \mathrm{e}^{\beta_j/\epsilon - 1/2}$ (any split of the constant e^{-1} between them will do). The entropy term keeps every $\gamma_{ij} > 0$, so the constraint $\gamma \geq 0$ never binds